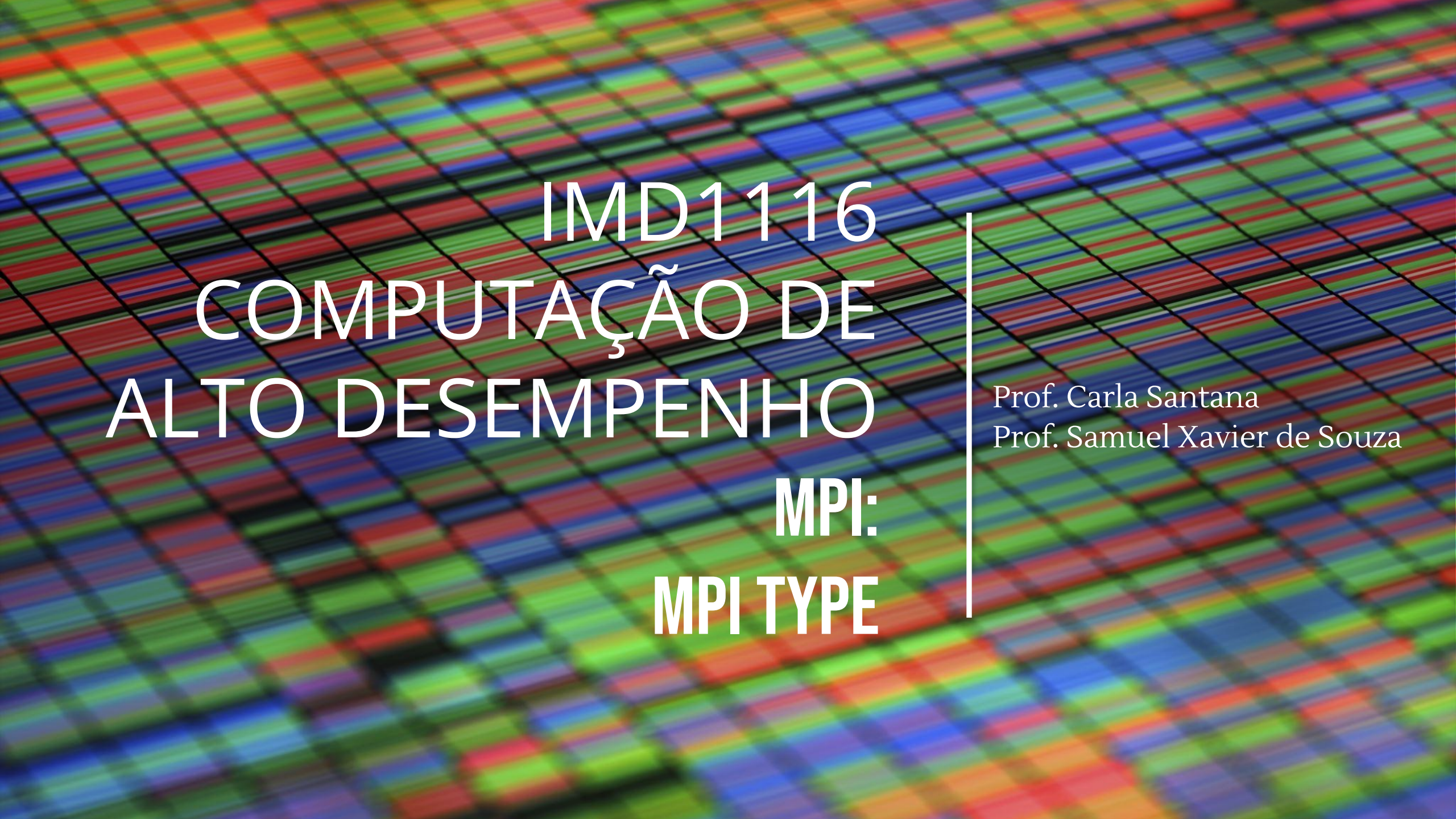




IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MPI:

MPI TYPE

Prof. Carla Santana
Prof. Samuel Xavier de Souza

Agrupar dados para comunicação

- Maximizar performance → Minimizar a troca de mensagens

Agrupar dados para comunicação

- Maximizar performance → Minimizar a troca de mensagens



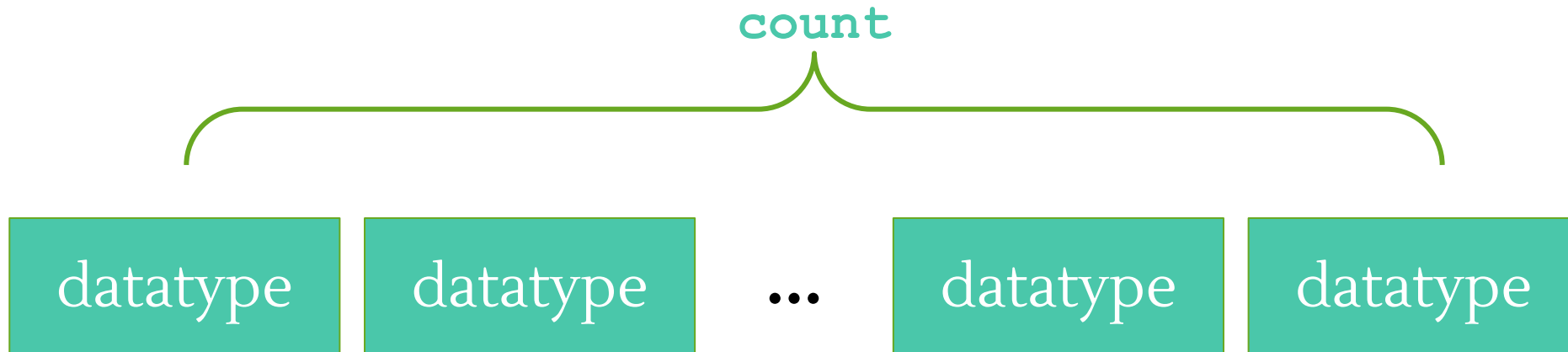
```
double x[1000];  
...  
if (my rank == 0)  
    for (i = 0; i < 1000; i++)  
        MPI Send(&x[i], 1, MPI DOUBLE, 1, 0, comm);  
else /* my rank == 1 */  
    for (i = 0; i < 1000; i++)  
        MPI Recv(&x[i], 1, MPI DOUBLE, 0, 0, comm, &status);
```



```
if (my rank == 0)  
    MPI Send(x, 1000, MPI DOUBLE, 1, 0, comm);  
else /* my rank == 1 */  
    MPI Recv(x, 1000, MPI DOUBLE, 0, 0, comm, &status);
```

Agrupar dados para comunicação

- Maximizar performance → Minimizar a troca de mensagens
- Por padrão, todas as funções de envio e recepção de mensagens permitem agrupar numa mesma mensagem dados do mesmo tipo guardados em posições contíguas de memória.

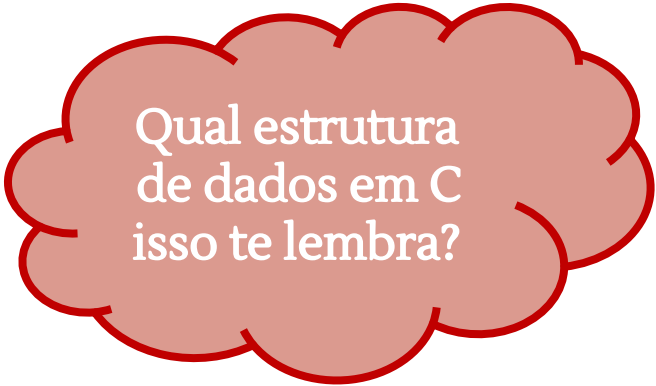


Agrupar dados para comunicação

- Maximizar performance → Minimizar a troca de mensagens
 - Por padrão, todas as funções de envio e recepção de mensagens permitem agrupar numa mesma mensagem dados do mesmo tipo guardados em posições contíguas de memória.
 - Para além desta funcionalidade básica o MPI permite:
 - Definir **novos tipos de dados** que agrupam dados de vários tipos.
 - **Empacotar e desempacotar dados para/de um buffer**
 - **Tipos Derivados**
-

Tipos Derivados

- Permitem aos programadores descrever layouts de dados complexos na memória e comunicá-los de forma eficiente entre processos.
- Útil quando os dados a serem enviados ou recebidos **não são contíguos na memória** ou quando se deseja **agrupar diferentes tipos de dados em uma única operação de comunicação** .



Qual estrutura de dados em C isso te lembra?

Tipos Derivados

- **Eficiência** : Evitam a necessidade de empacotar e desempacotar dados manualmente, o que pode ser custoso em termos de desempenho. O MPI pode otimizar a comunicação de dados não contíguos.
 - **Conveniência** : Simplificam o código, permitindo que o programador trate um conjunto complexo de dados como uma única unidade para fins de comunicação.
 - **Flexibilidade** : Permitem a comunicação de estruturas de dados heterogêneas, seções de arrays multidimensionais, ou qualquer padrão de dados não contíguo.
-

Tipos Derivados

- A definição de novos tipos de dados **é feita em tempo de execução** :
 - Inicialmente, os processos devem construir o novo tipo derivado.
 - A construção de tipos derivados é feita a partir dos tipos de dados básicos que o MPI define (`MPI_INT`, `MPI_FLOAT` ...).
 - Em seguida, devem certificar perante o ambiente de execução do MPI a existência do novo tipo derivado.
 - Após a sua utilização, cada processo libera a certificação feita.
-

Tipos Derivados

- **A construção de tipos derivados é dispendiosa**, pelo que só deve ser utilizada quando o número de mensagens a trocadas é significativo

Tipos Derivados

- **A construção de tipos derivados é dispendiosa** , pelo que só deve ser utilizada quando o número de mensagens a trocadas é significativo
 - Os novos tipos de dados são utilizados nas funções de envio e recepção de mensagens tal como os restantes tipos básicos.
 - Necessário que ambos os processos emissor e receptor tenham certificado o novo tipo derivado.
 - **Normalmente, isso é feito na parte do código que é comum a ambos os processos**
-

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em `count` **blocos**
- Cada bloco contém `blocklength` cópias de `oldtype`.
- Os blocos são separados por um `stride` (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

MPI_Type_vector

```
int MPI_Type_vector(int count, int blocklength, int stride, MPI_Datatype  
oldtype, MPI_Datatype *newtype)
```

- **count** : Número de blocos.
 - **blocklength** : Número de elementos em cada bloco.
 - **stride** : e o número de elementos contíguos que separa o início de cada bloco (deslocamento).
 - **oldtype** : Tipo de dado MPI existente.
 - **newtype** : Handle para o novo tipo de dado derivado.
-

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em `count` **blocos**
- Cada bloco contém `blocklength` cópias de `oldtype`.
- Os blocos são separados por um `stride` (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

```
#define ROWS 4  
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em `count` **blocos**
- Cada bloco contém `blocklength` cópias de `oldtype`.
- Os blocos são separados por um `stride` (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

```
#define ROWS 4  
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12


```
int MPI_Type_vector(int count, int blocklength, 3, MPI_INT, MPI_Datatype *newtype)
```

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em `count` **blocos**
- Cada bloco contém `blocklength` cópias de `oldtype`.
- Os blocos são separados por um `stride` (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

```
#define ROWS 4  
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

```
int MPI_Type_vector(int count, 1, 3, MPI_INT, MPI_Datatype *newtype)
```

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em `count` **blocos**
- Cada bloco contém `blocklength` cópias de `oldtype`.
- Os blocos são separados por um `stride` (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

```
#define ROWS 4  
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

```
int MPI_Type_vector(4, 1, 3, MPI_INT, MPI_Datatype *newtype)
```

MPI_Type_vector

- constrói um novo tipo de dados a partir de um vetor de dados
- Cria um tipo de dado que consiste em count **blocos**
- Cada bloco contém **blocklength** cópias de `oldtype`.
- Os blocos são separados por um **stride** (passo) de elementos do `oldtype`.
- Útil para acessar colunas de uma matriz ou sub-arrays.

```
#define ROWS 4  
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

```
#include <mpi.h>
#include <stdio.h>

#define ROWS 4
#define COLS 3

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype column_type;
    MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &column_type);
    MPI_Type_commit(&column_type);

    if (rank == 0) {
        int matrix[ROWS][COLS] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12}
        };
        // Enviar a segunda coluna (índice 1)
        MPI_Send(&matrix[0][1], 1, column_type, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou a coluna: %d %d %d %d\n", matrix[0][1],
matrix[1][1], matrix[2][1], matrix[3][1]);
    } else if (rank == 1) {
        int received_column[ROWS];
        MPI_Recv(received_column, ROWS, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu a coluna: %d %d %d %d\n",
received_column[0], received_column[1], received_column[2],
received_column[3]);
    }

    MPI_Type_free(&column_type);
    MPI_Finalize();
    return 0;
}
```

```
#define ROWS 4
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

```
#include <mpi.h>
#include <stdio.h>

#define ROWS 4
#define COLS 3

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype column_type;
    MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &column_type);
    MPI_Type_commit(&column_type);

    if (rank == 0) {
        int matrix[ROWS][COLS] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12}
        };
        // Enviar a segunda coluna (índice 1)
        MPI_Send(&matrix[0][1], 1, column_type, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou a coluna: %d %d %d %d\n", matrix[0][1],
matrix[1][1], matrix[2][1], matrix[3][1]);
    } else if (rank == 1) {
        int received_column[ROWS];
        MPI_Recv(received_column, ROWS, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu a coluna: %d %d %d %d\n",
received_column[0], received_column[1], received_column[2],
received_column[3]);
    }

    MPI_Type_free(&column_type);
    MPI_Finalize();
    return 0;
}
```

```
#define ROWS 4
#define COLS 3
```

1	2	3
4	5	6
7	8	9
10	11	12

```

#include <mpi.h>
#include <stdio.h>

#define ROWS 4
#define COLS 3

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype column_type;
    MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &column_type);
    MPI_Type_commit(&column_type);

    if (rank == 0) {
        int matrix[ROWS][COLS] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12}
        };
        // Enviar a segunda coluna (índice 1)
        MPI_Send(&matrix[0][1], 1, column_type, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou a coluna: %d %d %d %d\n", matrix[0][1],
matrix[1][1], matrix[2][1], matrix[3][1]);
    } else if (rank == 1) {
        int received_column[ROWS];
        MPI_Recv(received_column, ROWS, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu a coluna: %d %d %d %d\n",
received_column[0], received_column[1], received_column[2],
received_column[3]);
    }

    MPI_Type_free(&column_type);
    MPI_Finalize();
    return 0;
}

```

```

#define ROWS 4
#define COLS 3

```

1	2	3
4	5	6
7	8	9
10	11	12

```

#include <mpi.h>
#include <stdio.h>

#define ROWS 4
#define COLS 3

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype column_type;
    MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &column_type);
    MPI_Type_commit(&column_type);

    if (rank == 0) {
        int matrix[ROWS][COLS] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12}
        };
        // Enviar a segunda coluna (índice 1)
        MPI_Send(&matrix[0][1], 1, column_type, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou a coluna: %d %d %d %d\n", matrix[0][1],
matrix[1][1], matrix[2][1], matrix[3][1]);
    } else if (rank == 1) {
        int received_column[ROWS];
        MPI_Recv(received_column, ROWS, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu a coluna: %d %d %d %d\n",
received_column[0], received_column[1], received_column[2],
received_column[3]);
    }

    MPI_Type_free(&column_type);
    MPI_Finalize();
    return 0;
}

```

```

#define ROWS 4
#define COLS 3

```

1	2	3
4	5	6
7	8	9
10	11	12


```

#include <mpi.h>
#include <stdio.h>

#define ROWS 4
#define COLS 3

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype column_type;
    MPI_Type_vector(ROWS, 1, COLS, MPI_INT, &column_type);
    MPI_Type_commit(&column_type);

    if (rank == 0) {
        int matrix[ROWS][COLS] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12}
        };
        // Enviar a segunda coluna (índice 1)
        MPI_Send(&matrix[0][1], 1, column_type, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou a coluna: %d %d %d %d\n", matrix[0][1],
matrix[1][1], matrix[2][1], matrix[3][1]);
    } else if (rank == 1) {
        int received_column[ROWS];
        MPI_Recv(received_column, ROWS, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu a coluna: %d %d %d %d\n",
received_column[0], received_column[1], received_column[2],
received_column[3]);
    }

    MPI_Type_free(&column_type);
    MPI_Finalize();
    return 0;
}

```

MPI Dados derivados

Atv Extra - Adapte o exemplo ao lado para enviar a 3ª coluna, depois faça outra versão para enviar as colunas 2 e 3 ao mesmo tempo.

Atividade para ser entregue até **17:30h**.



MPI_Type_create_struct

- Cria um tipo de dado a partir de uma lista arbitrária de tipos de dados, blocos e deslocamentos em bytes.
- É a função mais flexível e poderosa para criar tipos de dados derivados, especialmente para estruturas (structs) em C

MPI_Type_create_struct

```
int MPI_Type_create_struct(int count, const int array_of_blocklengths[], const MPI_Aint  
array_of_displacements[], const MPI_Datatype array_of_types[], MPI_Datatype newtype)
```

- **count** : Número de blocos (membros da estrutura).
 - **array_of_blocklengths** : Array de inteiros, onde `array_of_blocklengths[i]` é o número de elementos no i-ésimo bloco.
 - **array_of_displacements** : Array de MPI_Aint (endereços), onde `array_of_displacements[i]` é o deslocamento em bytes do i-ésimo bloco em relação ao início do buffer.
 - **array_of_types** : Array de MPI_Datatype , onde `array_of_types[i]` é o tipo de dado do i-ésimo bloco.
 - **newtype** : Handle para o novo tipo de dado derivado (identificador/referência abstrata usada para acessar algum recurso sem expor diretamente sua implementação interna).
-

```
typedef struct {
    int id;
    double value;
    char name[10];
} MyData;

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Datatype mydata_type;
    int blocklengths[3] = {1, 1, 10}; // id (1 int), value (1 double),
(10 chars)
    MPI_Datatype types[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR};
    MPI_Aint offsets[3];
```

```
// Calcular os deslocamentos dos membros da estrutura
MyData dummy_data;
MPI_Get_address(&dummy_data.id, &offsets[0]);
MPI_Get_address(&dummy_data.value, &offsets[1]);
MPI_Get_address(&dummy_data.name, &offsets[2]);

// Ajustar os deslocamentos para serem relativos ao início da estrutura
MPI_Aint base_address;
MPI_Get_address(&dummy_data, &base_address);
for (int i = 0; i < 3; i++) {
    offsets[i] -= base_address;
}

MPI_Type_create_struct(3, blocklengths, offsets, types, &mydata_type);
MPI_Type_commit(&mydata_type);
```

Estrutura de um Tipo de Dado Derivado

Cada tipo de dado derivado é caracterizado por:

- **Tipo de Elemento (Typemap)** : Uma lista de pares (tipo básico MPI, deslocamento em bytes) que descreve os componentes do tipo de dado e suas posições relativas na memória..
- **Lower Bound (Limite Inferior)** : O menor deslocamento em bytes de um elemento em relação ao início do tipo de dado.
- **Upper Bound (Limite Superior)** : O maior deslocamento em bytes de um elemento em relação ao início do tipo de dado.
- **Extent (Extensão)**: O tamanho total em bytes ocupado pelo tipo de dado, incluindo quaisquer lacunas (**padding**) entre os elementos. É o deslocamento do primeiro byte do primeiro elemento até o primeiro byte após o último elemento

Padding

Padding são espaços “extras” inseridos na memória entre variáveis ou campos de uma estrutura para atender requisitos de alinhamento do hardware.

Exemplo:

char = 1 byte

int = 4 bytes

struct Exemplo {

 char a;

 int b;

}; $\Rightarrow$ Tamanho = 5 bytes?

Padding

Padding são espaços “extras” inseridos na memória entre variáveis ou campos de uma estrutura para atender requisitos de alinhamento do hardware.

Exemplo:

char = 1 byte

int = 4 bytes

struct Exemplo {

 char a;

 int b;

}; ⇒ Tamanho = 5 bytes?

sizeof(struct Exemplo) == 8

Padding

Padding são espaços “extras” inseridos na memória entre variáveis ou campos de uma estrutura para atender requisitos de alinhamento do hardware.

Exemplo:

char = 1 byte

int = 4 bytes

struct Exemplo {

 char a;

 int b;

}; ⇒ Tamanho = 5 bytes?

sizeof(struct Exemplo) == 8

Byte: 0 1 2 3 4 5 6 7
 a P P P b b b b

Padding

Se um dado estiver desalinhado:

- Acesso pode ficar mais lento
 - ou até causar erro em algumas arquiteturas
 - Então o compilador adiciona bytes “vazios”
-
- int ocupa 4 bytes
 - Muitos processadores acessam um int mais rapidamente quando ele começa em endereços múltiplos de 4

Padding

Se um dado estiver desalinhado:

- Acesso pode ficar mais lento
 - ou até causar erro em algumas arquiteturas
 - Então o compilador adiciona bytes “vazios”
-
- int ocupa 4 bytes
 - Muitos processadores acessam um int mais rapidamente quando ele começa em endereços múltiplos de 4

Imagine:

- um caminhão (int) precisa estacionar em vagas grandes,
- mas uma moto (char) pode parar em qualquer lugar.

Se a moto ocupa só uma parte:

- o sistema deixa espaço vazio
- para o caminhão estacionar alinhado.

Data Structure Alignment

Size of 1 block = 1 byte

Size of 1 row = 4 byte

a	b	b	c
c	c	c	d

Size of 1 block = 1 byte

Size of 1 row = 4 byte

a	padding	b	b
c	c	c	c
d	padding	padding	padding

MPI_Type_create_resized

- A função MPI_Type_create_resized é uma ferramenta avançada no MPI que **permite modificar o limite inferior (lower bound) e a extensão (extent) de um tipo de dado derivado existente.**
- MPI_Type_create_resized permite que você defina explicitamente o lower bound e o extent de um tipo de dado, garantindo que ele se comporte como esperado ao ser replicado

MPI_Type_create_resized

- Às vezes o extent calculado automaticamente pelo MPI pode não ser o ideal para **certas operações**, especialmente quando se deseja tratar um tipo de dado como um "bloco" de um tamanho específico para comunicação de arrays.
- Em cenários **como a criação de um array de estruturas onde cada estrutura tem um extent que não corresponde ao seu sizeof (devido a padding interno)**, o uso direto do tipo de dado derivado para enviar múltiplos elementos pode resultar em comportamento inesperado ou incorreto.

MPI Struct

Atv Extra - Implemente um programa MPI onde:

O processo 0 cria um vetor de estruturas representando partículas, cada estrutura possui:

- identificador,
- posição 2D,
- velocidade,
- massa;

o processo 0 envia as partículas para o processo 1 usando um datatype derivado MPI; o processo 1 recebe os dados e imprime as informações recebidas.

Atividade para ser entregue até **18:20h**.



Tarefa 18

Reimplemente a tarefa 17, agora distribuindo as colunas entre os processos.

Utilize apenas em uma versão `MPI_Type_vector` e em outra acrescente `MPI_Type_create_resized` para definir um tipo derivado que represente colunas da matriz.

Use `MPI_Scatter` com esse tipo para distribuir blocos de colunas, e `MPI_Scatter` para enviar os segmentos correspondentes de x .

Cada processo deve calcular uma contribuição parcial para todos os elementos de y e usa `MPI_Reduce` com `MPI_SUM` para somar os vetores parciais no processo 0.

Discuta as diferenças de acesso à memória e desempenho em relação à distribuição por linhas. Analise o desempenho das versões da tarefa 17, e das duas dessa tarefa.

Referências

